

סיכום קריפטולוגיה

דרישות לתקשורת מוצפנת :

1. **Authentication וודאות**: ברגע שאלים שלחה הודעה צריך לוודא שזו באמת אליס
2. **Integrity מקוריות**: צריך לוודא שההודעה היא באמת ההודעה שנשלחה ולא בוצעו בה שינויים
3. **Nonrepudiation אי הכחשה**: מניח הכחשת ההודעה, ללא תלות בזמן שעבר.

Eavesdropping האזנה:

1. האזנה פסיבית : התוקף רק מאזין למידע, גונב את המידע ולא משנה אותו
2. האזנה אקטיבית: התוקף מאזין ומשנה את הטקסט

Encryption-Decryption הצפנה ופענוח: לרוב אלגוריתמי הצפנה והפענוח הם פומביים וידועים לכל. מה שסודי הוא המפתח

1. אלגוריתמים סימטריים : מפתחות ההצפנה והפענוח זהים ושניהם סודיים
2. אלגוריתמים א-סימטריים : מפתח ההצפנה פומבי, מפתח הפענוח סודי.

דרישות:

1. אלגוריתם הפענוח E צריך להיות הפיך לכל מפתח
2. לכל מפתח הצפנה קיים מפתח פענוח
3. אם מפתחות ההצפנה שונים אז ההצפנה עבור כל טקסט יביא טקסט מוצפן שונה

Crypto System : מערכת קריפטולוגיה אשר בנויה מ:

- קבוצה סופית של טקסטים בשפה
- קבוצה סופית של טקסטים מוצפנים
- קבוצת מפתחות סופית
- דרישות הפענוח (למעלה)

Caesar Cipher או Shift Cipher: כל אות מוזזת k צעדים

חולשה: כל אחד שיוודע את ההזזה יכול לפענח כל הודעה.

מתקפות על Shift Cipher:

- התקפה ישירה Brute force: חיפוש כל האפשרויות של הזזה (קל כיוון שיש רק 26 הזזות אפשריות)
- התקפה לפי הסתברות Letter frequency analysis: ידוע באיזה מילים משתמשים הכי הרבה וזה יכול לתת רמז עבור איזה אותיות שונות.
- **Kerkhof's Principle** חוק קרקוף : מה יכול להבטיח את בטיחות התוצאה :

-אורך המפתח

-מגוון המפתחות

אנחנו בטוחים כאשר:

1. מחזיר פענוח האלגוריתם גדול בהרבה ממחזיר הצפנת הטקסט
2. מחזיר הזמן הפענוח גדול מזמן ההצפנה
3. כמות המידע שניתן לפענח בעזרת מפתח יחיד קטן מכמות המידע הכולל

Restricted Algorithms אלגוריתמים מוגבלים:

אלגוריתמים שקיימת עליהם הגבלה מסוימת לרוב הגבלה על מכשיר מסוים כמו מכונת אנגימה.

Symmetric Algorithms אלגוריתמים סימטריים:

מפתח ההצפנה ומפתח הפענוח סודיים

בעיה: צריך להעביר איכשהו את המפתח הפענוח לצד השני

סוגי אלגוריתמים סימטריים:

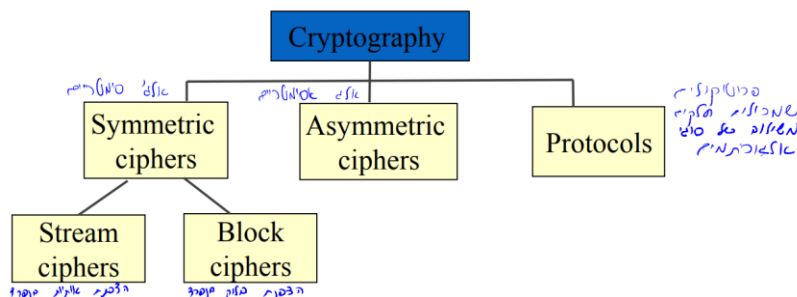
1. **Stream algorithms** : מצפינים כל ביט או בית
2. **Block algorithms** : מצפינים כל בלוק של בתים

Asymmetric Algorithms אלגוריתמים א-סימטריים : מפתח ההצפנה הוא פומבי מפתח הפענוח הוא סודי

אסימטריים פותרים את בעיית העברת המפתח (אך לרוב הם איטיים יותר)

Hybrid Algorithms משולבים : למשל ההצפנה בעזרת אלגוריתם סימטרי אבל העברת המפתח מתבצעת בעזרת הצפנה המפתח באלגוריתם אסימטרי.

Classification



One Time Pad : שיטה שפותרת את העברת המפתח באלגוריתמים סימטריים כאשר בכל פעם לפי מוסכמה מסוימת יודעים איזה מפתח הבא משתמשים, לרוב לפי רשימה ידועה מראש או על סמך המפתח הקודם ואז מתקדמים.

Random Number Generator : דוגמא ל'One time pad' כאשר יוצרים מפתח רנדומלי לפי הזרעים (seeds) שהוכנסו בהתחלה

Linear Affine Cipher אלגוריתם ההצפנה הפיני: ההצפנה בנויה משני ערכים (a, b) : $E_k(x) = (ax + b) \bmod(n)$ כאשר מותר רק ערכי a שזורים ל n . קיימת בעיה בפענוח שיכולות להתקבל שתי תוצאות זהות עבור טקסטים שונים ז"א לא תח"ע.

Monoalphabetic Substitution Ciphers הפנה על בסיס פרמוטציות: קיימים 26! אפשרויות להצפנת האותיות

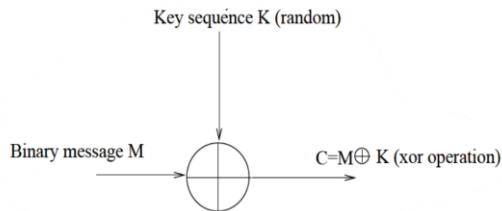
Brute Force – ניסיון של כל המפתחות עד לקבלה (ניתן לקצר על ידי בדיקת אותיות שחוזרות או רווחים)

לרוב ניתן לפרוץ את הצפנים הקלאסיים על ידי סטטיסטיקה של המופעים של האותיות

Stream Ciphers : הצפנות של ביטים. עוברים כעל כל ביט ומצפינים אותו.

סוגי הצפנות של Stream Ciphers :

Vernam Cipher : גודל המפתח בגודל של הטקסט כאשר עושים XOR בין המפתח לטקסט



encryption: $E_k(x) = (x \oplus k)$

decryption: $D_k(y) = (y \oplus k)$

where $x, y, k \in \mathbb{Z}_2$.

Cryptanalysis : בוחנים את תהליך ההצפנה מצד התוקף. נקודת המוצא שהתוקף לא יודע את המפתח וצריך לחשוב איך התוקף היה מבצע את

התקיפה

Ciphertext Only Attack : לתוקף יש רק את ההודעה המוצפנת או מספר הודעות מוצפנות, ואם הוא יגלה את המפתח הוא יכול לפרוץ

Known Plaintext Attack : לתוקף יש גם את הטקסט המוצפן וגם את המקורי. אבל זה לא תמיד עוזר להודעה שמוצפנת עכשיו.

Chosen Plaintext Attack : יש רק את החלק של ההודעה המקורית של חלק וחלק של הטקסטים המוצפנים.

Chosen Ciphertext Attack : יש רק חלקים של הטקסט המוצפן

Rubber Hose Cryptanalysis : השגת מפתח בצורות אלימות.

Security of Algorithms : אבטחת אלגוריתמי ההצפנה

- אלגוריתם ייקרא מוצפן ללא תנאי unconditionally secure אם הוא לא ניתן לפריצה לא משנה כמה טקסטים מוצפנים יש לו (ז"א הצפנה כל אות עם מפתח חדש). רק **One Time Pad** הם מוצפנים ללא תנאי!! הדרך היחידה לפצח הוא לנסות בעזרת Brute force כאשר ההתקפה הזאת טובה אבל מאד איטית.

- אלגוריתם ייקרא מוצפן חישובית computationally secure אם לא ניתן לפרוץ אותו בעזרת המשאבים הקיימים. (כיוון שזה ייקח אלפי שנים)

מה צריך לקחת בחשבון בחישוב סיבוכיות ההתקפה: אם עלות ההתקפה גבוהה מערך המידע שנקבל אז זה לא רלוונטי

- כמות המידע המינימאלי שצריך כדי לבצע את הפריצה

-הזמן הדרוש לביצוע ההתקפה

-כמות המקום שנדרש לצורך ביצוע ההתקפה

התקפה סטטיסטית : משתמשים בסטטיסטיקת השפה לצורך ידיעת מילות ידועות

התקפה איטרטיבית : יש טקסט מוצפן נצפין אותו n פעמים עד קבלת אותו טקסט מוצפן ואז נדע שבאיטרציה $n-1$ זה הטקסט המקורי

ז"א כאשר ההצפנה היא קופסא שחורה ומשתמשים על הצפנים האסימטריים. כיוון שבאלגוריתמים הסימטריים ניתן לדעת את אופן ההצפנה ההתקפה היא בסדר גודל של Brute Force.

הודעה בלתי מוסתרת Unhidden messages : עבור הודעה x אם מצפינים ומקבלים את אותו x ז"א : $E_k(x) = x$ ז"א קיים מפתח שנותן את אותה ההודעה.

עבור כל צופן : נתבונן בכמות ההודעות הבלתי מוסתרות שיש. אם נקבל כמות גדולה אז הצופן לא טוב.

Block ciphers : אלגוריתם שמצפין בלוקים של אותיות או מספרים ביחד

Hill cipher : הצפנה בעזרת מטריצת $m \times m$ כאשר מצפינים m אותיות בו זמנית

כל טקסט ניתן לחלק לבלוקים בגודל ידוע של m .

$$E_k(\vec{x}) = \vec{x} \cdot K \bmod n$$

$$D_k(\vec{y}) = \vec{y} \cdot K^{-1} \bmod n$$

$$\text{possible if } \exists K^{-1} \bmod n \Leftrightarrow \gcd(\det(K), n) = 1$$

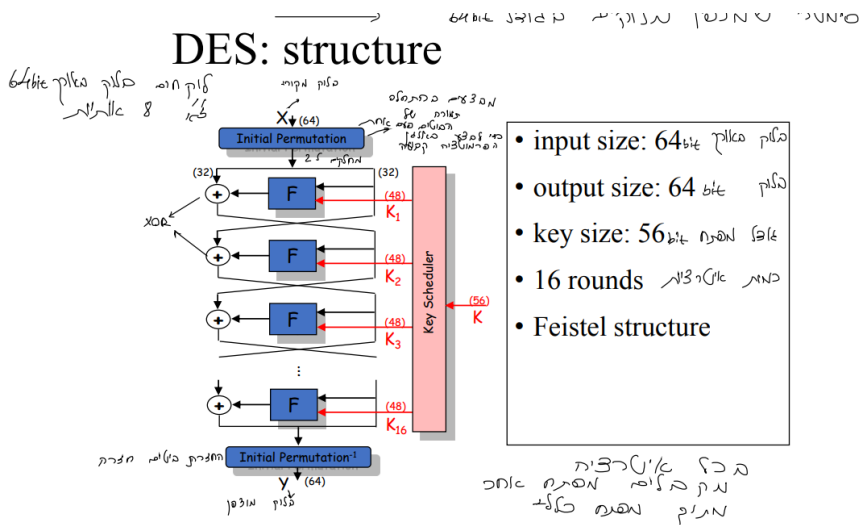
זאת אומרת נדרוש שמפתח (המטריצה) תהיה הפיכה מודולו n

Permutation Cipher = Hill Cipher: הצפנת פרמוטציה בצורת צופן היל, כאשר המפתח יהיה מטריצת פרמוטציה חולשות אלגוריתם היל: קל לפרוץ בעזרת טקסט ידוע שקיים (גם את הצפנת פרמוטציה וגם היל)

The Permutation Cipher: לא נגדיר נוסחה לכל צופן, בכל פעם נציין בטבלה אחרת

The Vigenere Cipher: כל אות מוזזת לפי המפתח שלה, נגדיר לכל מילה מפתח אחר כאשר כל אות תהיה מוזזת בכמות אחרת.

DES: structure



אלגוריתם DES :

סוג: אלגוריתם סימטרי שמצפין בלוקים

גודל מפתח: $|k| = 56 \text{ bit}$ (גודל של 56 ביטים)

ביסוס: רשת פייסטל Feistel scheme (חומרה – מעגל)

תהליך האלגוריתם:

1. מקבל בלוק בגודל 64 ביט (8 שמיניות)

2. מבצעים מטריצת פרמוטציה ראשונית

3. מחלקים את הבלוק לצד שמאל L וצד ימין R

4. מבצעים 16 איטרציות בהם:

4.1 מבצעים פרמוטציה Permutation

4.2 מבצעים החסרה Substitution

4.3 חלק ימין הולך לשמאל וחלק שמאל הולך לימין

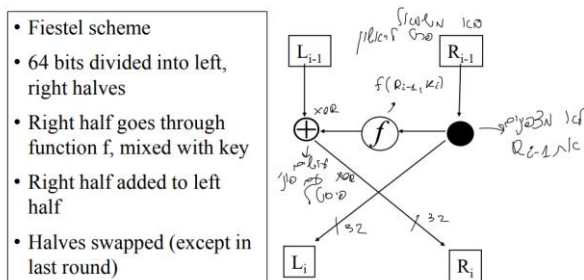
איטרציה i של DES :

$$f(R_{i-1}, k_i) \text{ ו-} k_i \text{ מפתח יחד עם פייסטל } R_{i-1}.$$

ובנוסף הופך להיות שמאל הבא $R_{i-1} \rightarrow L_i$

על L_{i-1} מבוצע XOR עם התוצאה שקיבלנו מביצוע פייסטל

זהה הולך להיכנס בתור ימין $L_{j-1} \oplus f(R_{j-1}, k_j) \rightarrow R_j$



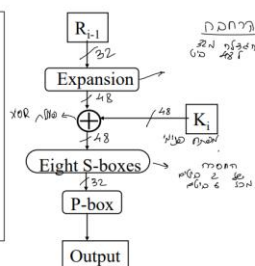
פונקציית פייסטר :

Expansion - : נחלק את הבלוק של 32 ביטים לרביעיות ונוסיף ביט מכל

צד של הרביעייה הבאה והקודמת

The DES round function f

- Expand right side from 32 to 48 bits (some get reused)
- Add 48 bits of key (chosen by schedule)
- S-boxes: each set of 6 bits reduced to 4 bits
- P-box permutes 32 bits



Expansion

32 1 2 3 4 5
 4 5 6 7 8 9
 8 9 10 11 12 13
 12 13 14 15 16 17
 16 17 18 19 20 21
 20 21 22 23 24 25
 24 25 26 27 28 29
 28 29 30 31 32 33

S-BOX

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

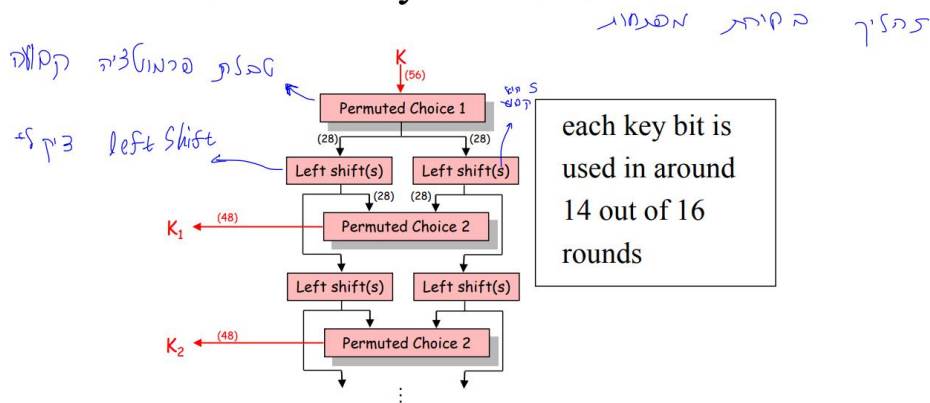
For example, for input 01101 the row is 01, that is row 1, and the column is determined by 1101, that is column 13.

In row 1 column 13 appears 5 so that the output is 0101.

P-Box - פרמוטציה בה לכל אינדקס מקצים אינדקס חדש

THE UNIVERSITY OF CHICAGO

The DES key scheduler



התקפות על DES :

התקפה ישירה : ניחוש מפתח בגודל של 56 ביט. 2^{56} מפתחות (בגלל עניין המשלים ל-2 קיימים רק 2^{55} מפתחות) : סוגים של הצפנת בלוקים :

ECB Electronic Code Book : משתמשים להצפנת בלוק יחיד (DES)

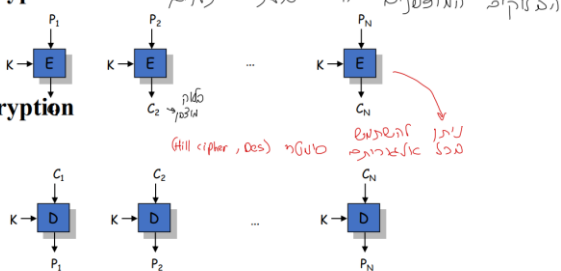
CBC Cipher Block Chaining : שימוש חוזר באותו אלגוריתם על בלוקים עד להצפנת הטקסט כולו

CFB Cipher Feedback : מצפינים ביטים או אותיות כאשר מצפינים כל אותו כאשר היא מגיעה (Stream Cipher)

OFB Output Feedback : כמו CFB עבור סטרים

ECB mode - Electronic Codebook

• encryption



• decryption

ECB – Electronic Code Book : כל בלוק מוצפן בעזרת אותו מפתח k . הבלוקים המוצפנים הם בלתי תלויים.

CBC mode – Cipher Block Chaining :

מחלקים את הטקסט לבלוקים ואת כולם מצפינים בעזרת מפתח כללי k , כל בלוק תלוי בזה שלפניו כאשר מפענחים צריכים לדעת את הבלוק הראשון כי בו תלויה ההצפנה והתלות

כל בלוק מוצפן תלוי בבלוק המוצפן הקודם

IV – initial Vector

חובה לשלוח IV, טקסט ומפתח

$\text{Encrypt}_{\text{CBC}}(\text{IV}, \text{Text}, \text{KEY})$,

$\text{Decrypt}_{\text{CBC}}(\text{IV}, \text{Cipher Text}, \text{KEY})$

* IV : בלוק ראשוני שלפיו מצפינים

** אם המפתח והטקסט אותו דבר אבל ה IV הראשוני שונה נקבל תוצאות

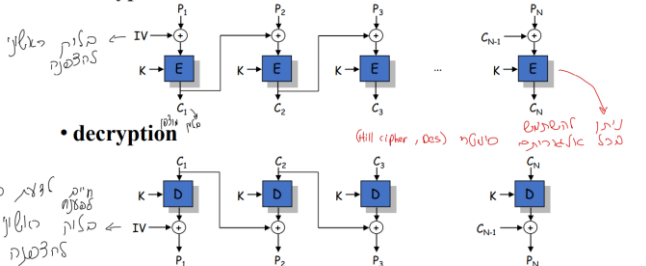
שונות בגלל התלות שקיימת בין הבלוקים

*** הבלוק IV לא צריך להיות סודי אבל הוא חייב להיות בעל מקוריות

(Integrity) זאת אומרת צריך להיות בטוחים שלא בוצעו בו שינויים.

CBC mode - Cipher Block Chaining

• encryption



חולשות של DES :

-מפתחות חלשים שלא באמת מצפינים
-כל הטבלאות קבועות וידועות מראש S-box, P-box הם לא באמת רנדומאליות

מפתחות חלשים: מפתח שאם נצפין פעמיים נקבל אותו טקסט (סה"כ 4

מפתחות חצי חלשים: זוגות של מפתחות שאם נצפין אותם ביחד נקבל את הטקסט המקורי. (סה"כ 6 זוגות של מפתחות)

***הצפנה חוזרת של DES :** הצפנה חוזרת לא מחלישה את האלגוריתם אך לא מובטח גם שהיא תמיד תעזור, (עבור הצפנה כפולה לא תמיד קיים מפתח שיצפין באותה הצורה)

התקפה נגד הצפנה כפולה של DES : MAN IN THE MIDDLE

נצפין את ההודעה המקורית P ונקבל את M

נפענח במקביל את ההודעה המוצפנת C ונקבל את M

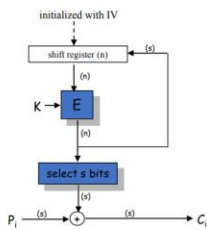
זאת אומרת שבמקום לפענח 2^{56} פעמים אנחנו נפענח פעם אחת ונצפין פעם אחת ונשווה את הטקסטים

מסקנה לא מחזק את האלגוריתם

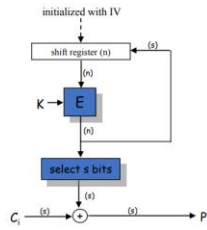
**** עובר הצפנות זוגיות ההצפנה לא מתחזקת אבל עובר הצפנות לא זוגיות כן למשל 3DES.**

OFB mode

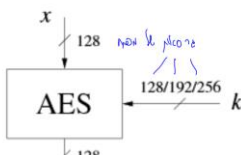
encrypt



decrypt

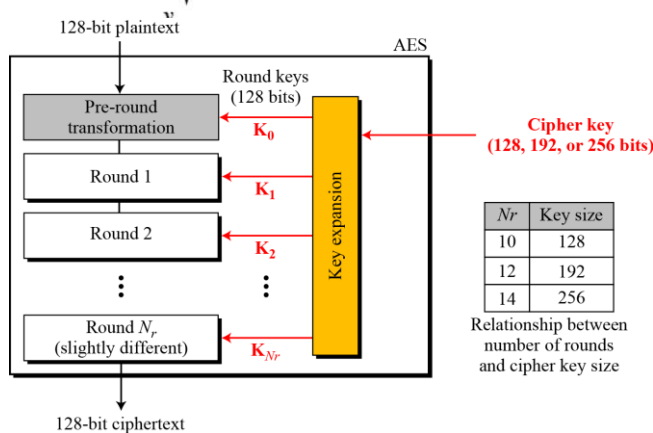


AES input/output parameters



Key lengths and number of rounds for AES

key lengths	# rounds = n_r
128 bit	10
192 bit	12
256 bit	14



Relationship between number of rounds and cipher key size

N_r	Key size
10	128
12	192
14	256

אלגוריתם AES : אלגוריתם סימטרי המצפין בלוקים בגודל 128bit

דרישות AES :

-הצפנה סימטרית

-מצפין בלוקים

-מצפין בלוקים בגודל 128bit

-חזק ומהיר יותר מ: 3Des

-קל לשימוש ומימוש

מימוש Rijndael ל AES :

גודל בלוק: 128bit

גודל מפתחות: 128/192/256 bit

ביסוס: מבוסס על איטרציות (לא מבוסס על רשת פייסטל)

ההצפנה מבוססת על שדה גבוה $GF(2^8)$ כאשר כל הפעולות על 8bit כאשר כל

מקדם של הפולינום מדרגה 8

ז"א עבור בלוק בגודל 8bit כאשר: $\{b_0, b_1, \dots, b_7\} \in \{0,1\}$

נקבל $f(x) = (b_7x^7 + b_6x^6 + \dots + b_1x + b_0) \bmod 2$

פעולות:

פעולת חיבור (XOR)

פעולת כפל רגילה תחת חילוק בפונקציה שלא ניתנת לפירוק

אלגוריתם Rijndael :

-מתקבל בלוק DATA בגודל 128bit

-מחלקים לטבלה של 4×4 בתים (4 שורות על 4 עמודות של 8bit בכל תא)

-איטרציות (10/12/14 כחלות בגודל המפתח), בכל איטרציה :

1. ביצוע החסרה של S-Box (לפי טבלאות קבועות) ניגש לאינדקס בטבלת ההחלפה

לפי הערך שלנו ונחליף עם הערך שבטבלה

2. ביצוע הזזה ציקלית Shift Rows שורה i תזוז i צעדים

3. שימוש במטריצות קבועות אשר מבלגנות את

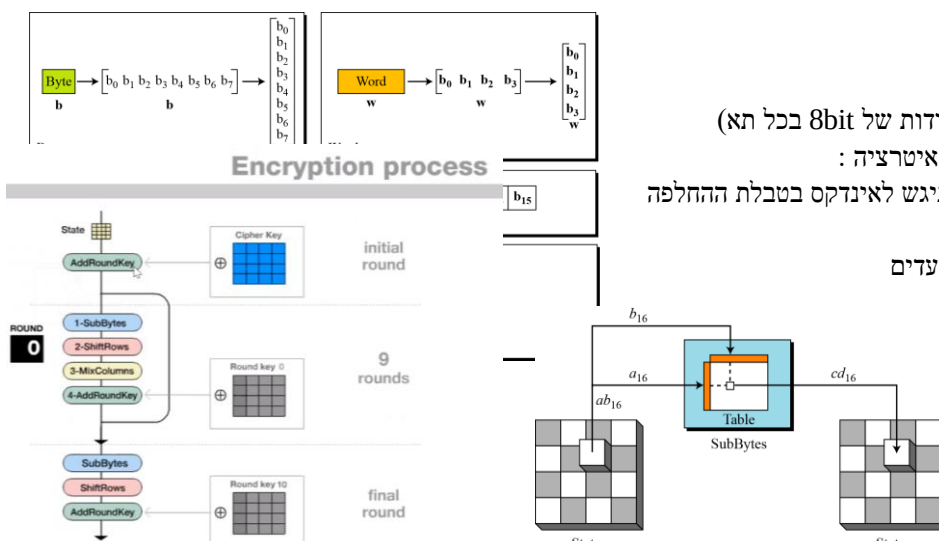
הנתונים Mixing (כפל בין המטריצה שלנו אל

מול הכפל) **מכפלה פונימיאלית**

4. הוספת מפתח מספר i לאיטרציה ה i

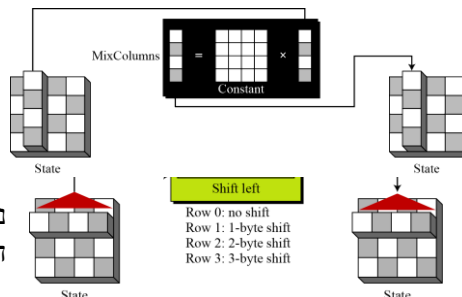
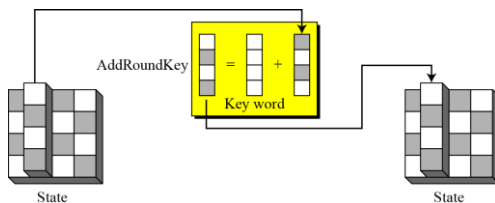
(באיטרציה האחרונה לא מבוצע כי מבוצע

בהתחלה):המפתח נגזר מהמפתח הכללי שקיבלנו

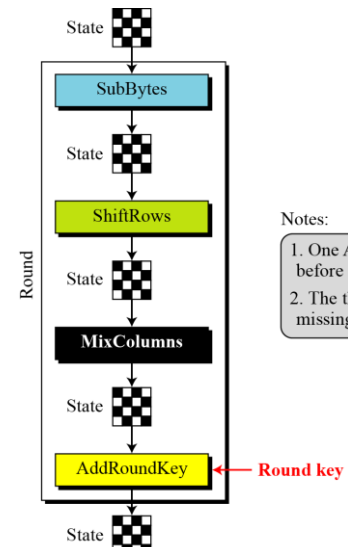


בניית מפתח עבור AES :

נשתמש בטבלת Rcon קבועה אשר תחילה
נבצע החלפה Subbytes ואז נבצע XOR עם
הטבלה



Shift left
Row 0: no shift
Row 1: 1-byte shift
Row 2: 2-byte shift
Row 3: 3-byte shift



Notes:

1. One AddRoundKey is applied before the first round.
2. The third transformation is missing in the last round.

תיאורית סודיות המערכת :

לפי התיאוריה של שאנון ניתן לבדוק האם ההצפנה

היא בעלת "סודיות מושלמת" וניתן לבדוק מתי היא בעלת "סודיות מושלמת"
הרצאה 4

העברת מפתח D-H Diffie-Hellman

בעזרת שיטה זו לא מצפינים אלא עבור אלגוריתמים סימטריים איך מעבירים מפתח הצפנה.
ז"א איך המצפין והמפענח מקבלים ומעבירים מפתח.

אלגוריתם D-H :

לכל אחד יש מפתח סודי

צריכים לבחור $g = \text{generator}$ פומבי (g הוא מפתח הצפנה פומבי)
האלגוריתם יהיו g, n, X, Y פומביים, כאשר n מספר ראשוני גדול

מאד, $0 \leq x, y \leq n - 1$

1. לאלים יש מפתח סודי x

לבוב יש מפתח סודי y

2. אלים מחשבת $X = g^x \mod(n)$

בוב מחשב $Y = g^y \mod(n)$

3. אלים שולח את X

בוב שולח את Y

4. אלים קיבלה Y ומצפינה עם המפתח שלה $K = (Y)^x \mod(n) = (g^y)^x \mod(n) = g^{xy} \mod(n)$

בוב קיבל X ומצפין עם המפתח שלו $K = (X)^y \mod(n) = (g^x)^y \mod(n) = g^{xy} \mod(n)$

הערות: עבור $X = g^x \mod(p)$

p - הוא ראשוני גדול מאד $2^{400} > p$, g מייצר

g, p הם פומביים

- אלים צריכה לבחור מפתח סודי $0 \leq x \leq p - 1$ ולשלוח לבוב

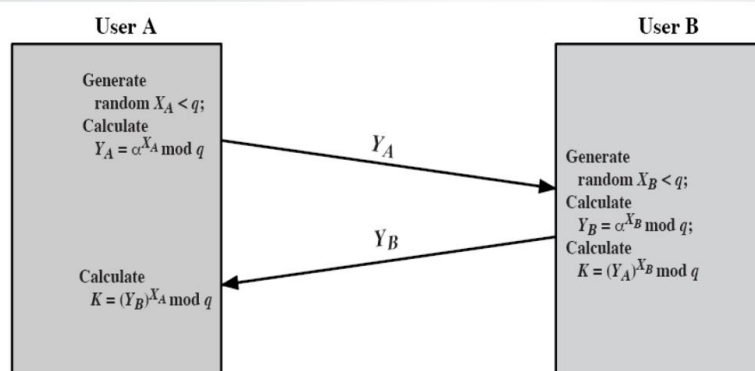
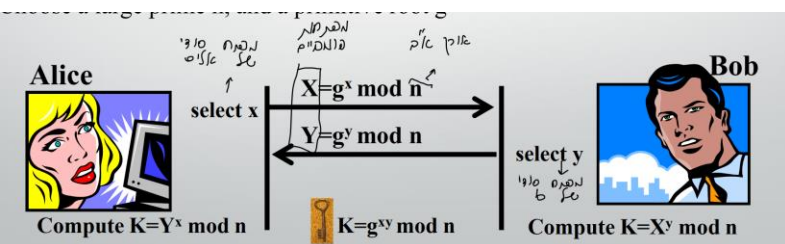
g^x כנל עבור בוב

- מי שמאזין יכול לקבל גם g^x וגם את g^y , אבל הוא לא יכול

לחשב את x או y .

בגלל שהמאזין לא יכול לחשב את x או את y אז הוא לא יכול

לחשב את המפתח k



חסרון של D-H :

הפרוטוקול לא נותן אימות לכן ניתן לבצע תקיפה של **Man In The Middle**
פתרון: ניתן לפרסם את (g, n, g^x) שהם פומביים (ז"א להראות כבר את התוצאה של הסודיות)
חשוב : לא להשתמש במפתחות שהם 0 או 1 : ז"א $x, y \neq 0, 1$ כי הם חלשים

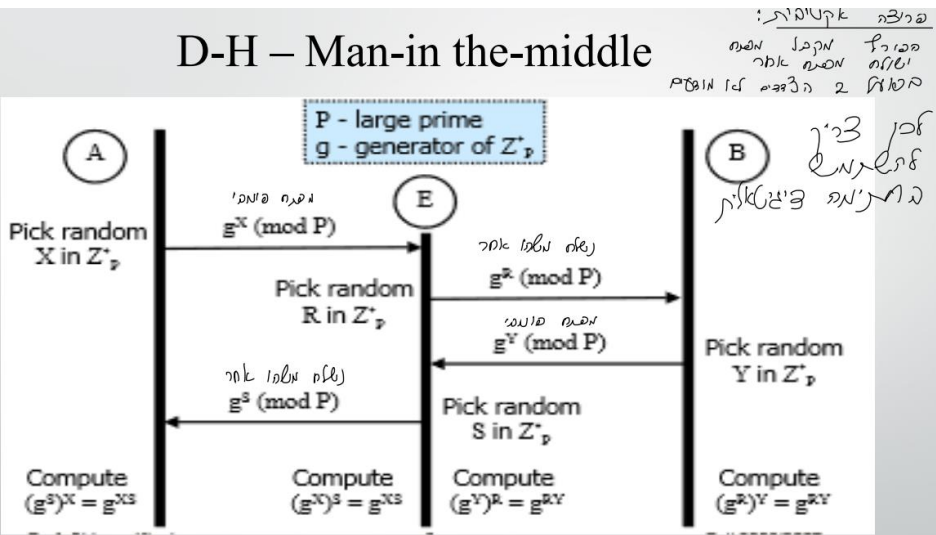
יתרונות של DH :

- כל אחד יכול לייצר מפתח x ולשלוח g^x
 - כל אחד יכול להצפין מידע באמצעות אותו g^x ולשלוח טקסט מוצפן לאותו בנאדם
 - אידיאלי עבור מיילים

חסרונות של DH :

- לא מגן נגד התקפת Man In The Middle , **פריצה**
אקטיבית: הפורץ מקבל מפתח ושולח מפתח אחר
 בפועל 2 הצדדים לא מודעים (לכן יש צורך להשתמש בחתימה דיגיטלית)
 - הפורץ מחליף את המפתח לאחד ושולח הלאה
 - עם g ניתן לפירוק לגורמים אז התוקף יכול למצוא את התת קבוצה היוצרת את המפתח וכיוון שלא יהיו הרבה צירופים הוא בקלות יוכל לזהות את המפתחות הסודיים

D-H – Man-in the-middle



RSA אלגוריתם אסימטרי: כולם יכולים להצפין רק אחד יכול לפענח

לכל צד יש זוג מפתחות (k_1, k_2) כאשר k_1 פומבי ומפתח k_2 סודי.

$$D_{k_2}(E_{k_1}(M)) = M$$

*אלגוריתמים אסימטריים הרבה יותר ארוכים בזמן חישוב לכן נשתמש עבור הודעות קצרות או נשתמש עבור הצפנת מפתח עבור הצפנות סימטריות.

רקע:

אופן פעולה: מקבל אבטחה כתוצאה של חזקות של מספרים גדולים.

האלגוריתם:

- מצפין בלוקים, כאשר כל בלוק הוא מספר בין 0 ל n כאשר $n = 2^k$

-הצד המקבל בוחר e, d

- n, e הם פומביים

- d הוא סודי

- הצפנה: $C = M^e \bmod(n)$

-פענוח: $M = C^d \bmod(n) = M^{ed} \bmod(n)$

דרישות:

- $0 \leq M \leq n - 1$

-צריך למצוא d, e כאלו שמקיימים $M = M^{ed} \bmod(n)$ לכל $M < n$

- צריך לוודא ש d לא ניתן לחישוב מתוך e, n גם אם הוא מחזיק בטקסט המוצפן

הקדמה:

- p, q צריכים להיות מספרים ראשוניים כך ש $p \cdot q = n$

- $Z_n^* = Z_{pq}^*$ מכיל את כל המספרים בטווח של $[1, pq-1]$ שהם ראשוניים ל p וגם עבור q

- הגודל הוא $|Z_{pq}^*| = \theta(pq) = (p-1)(q-1) = n - (p+q) + 1$

- כל $x \in Z_n^*$ מתקיים $x^{\theta(pq)} = x^{(p-1)(q-1)} = 1 \bmod(n)$

בפועל:

- כל אחד מייצר זוג מפתחות, אחד סודי ואחד פומבי על ידי בחירת 2 מספרים ראשוניים גדולים p, q .

-כל אחד מחשב $\theta(n) = (p-1)(q-1) : n = p \cdot q$

-צריך לבחור מספר רנדומאלי e כאשר $1 < e < \theta(n)$ וגם מתקיים $\gcd(e, \theta(n)) = 1$

-צריך לפתור את המשוואה למציאת מפתח הפענוח d כאשר $e \cdot d = 1 \bmod(\theta(n))$ וגם $d \in Z_n^*$

בפועל: $d = e^{-1} \bmod(\theta(n)) = e^{-1} \bmod(\theta(p) \cdot \theta(q)) = e^{-1} \bmod((p-1)(q-1))$

-כל אחד צריך לפרסם את מפתח ההצפנה הפומבי שלו $k_1 = \{e, n\}$

-כל אחד שומר לעצמו את מפתח הפענוח $k_2 = \{d, p, q\}$

סדר הפעולות: הסדר חשוב, מי רוצה להצפין הודעות?

נגיד אליס רוצה להצפין הודעות מבוב $A \rightarrow B$

1. A שולחת ל B בקשה לשליחת מפתח פומבי

2. B : בוחר p, q סודיים ומחשב את $n=pq$

$k_1 = \{e, n\}$ בחירת מפתח פומבי

בחירת מפתח סודי $k_2 = \{d, p, q\}$ (כאשר המפתח k_2 נגזר מתוך k_1)

מחשב: $k_1 \cdot k_2 = 1 \bmod(\theta(n))$

שולח ל A את $k_1 = \{e, n\}$ הפומביים

3. A מצפינה בעזרת המפתח k_1 את ההודעה כך שתקבל $C = M^e \bmod(n)$

A שולח את C המוצפן

4. B מפעיל את המפתח הסודי k_2 כך שמתקבל: $M = C^d \bmod(n) = (M^e)^d \bmod(n) = M^{(e \cdot d)} \bmod(n) = M^1 \bmod(n)$

**צריך להתקיים כי k_1 זר ל: $\theta(n)$ כדי שיהיה קיים הופכי

שימושים פרקטיים:

-הערכים p, q צריכים להיות מאד גדולים ומאותו סדר גודל

-generator הוא רנדומאלי "טוב": רנדומאלי ולא צפוי

- המחלק המשותף צריך להיות כמה שיותר קטן אחרת יהיו פחות ערכים ז"א $\gcd(p-1, q-1) = \text{small number}$

- לא משתמשים ב e שהוא זוגי

- d צריך להיות גדול משורש רביעי של n ז"א $d < n^{\frac{1}{4}}$

הודעות בלתי מוסרות ב RSA :

-כמה הודעות בלתי מוסרות : $M^e = M \bmod(n)$ יש ?

$$M^{(e-1)} = 1 \bmod(n) \rightarrow$$

$$1. M^{e-1} \bmod(p)$$

$$2. M^{e-1} \bmod(q)$$

$$(1 + \gcd(e-1, p-1)) \cdot (1 + \gcd(e-1, q-1)) \geq 9$$
 ז"א קיימים לפחות

$$\gcd(e-1, q-1) \geq 2$$
 כאשר

$$\gcd(e-1, p-1) \geq 2$$
 ו:

מכאן נקבל שההודעות הבלתי מוצפנות הטריטוריות הן כל השילובים של הזוגות של $(x, y) \in \{-1, 0, +1\}$

התקפות על RSA :

-לקיחת הרבה טקסטים מוצפנים ורגילים וניחוש d. מניעה : על ידי החלפת מפתחות באופן תדיר.

התקפה זו לוקחת המון זמן

-התקפה איטרטיבית: ספירת כמות האיטרציות עד קבלת אותה ההצפנה. בגלל זה במודולו אז נגיע בוודאות לאותה ההצפנה. לוקח המון זמן

דברים חשובים לקחת בחשבון:

- לא לחתום על הודעה רנדומאלית

- לחתום רק על hash

- להשתמש במפתחות שונים עבור חתימה דיגיטאלית והצפנת טקסט

-לא להשתמש באותו n עבור יותר ממשמש אחד

- עטיפה padding של ההודעה במספרים רנדומאליים צריך לבדוק ש M הוא באותו גודל של n

-לא לבחור במספרים נמוכים עבור e, d

פונקציות Hash : השימוש בהן נעשה עבור חתימות דיגיטליות

פונקציית Hash נותנת עבור כל גודל קלט נותנת פלט באורך קבוע

Many to one : הרבה קלטים יכולים ללכת לאותו הפלט, לכן יכולים להיות התנגשויות. (קשה למצוא את הקלטים שיביאו לנו את אותו הפלט) לכן לא ניתן לחשב את הערך המקורי של הקלט שהוכנס ז"א פונקציית hash היא חד סטרית

תכונות חשובות שצריך להיות עבור פונקציית Hash :

-קל לחישוב (סיבוכיות חישוב נמוכה)

-בלתי אפשרי חישובית למצוא קלט נוסף x' כך ש: $h(x') = h(x)$

-בלתי אפשרי חישובית למצוא שני קלטים שיש להם את אותו ערך של hash

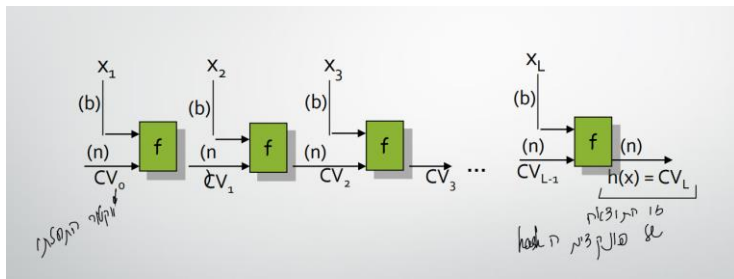
-חד ערכיות : בהינתן ערך $y=h(x)$ לא ניתן חישובית למצוא את ערך x

פרדוקס ימי ההולדת

בהינתן N פריטים, נבחר k פריטים, מה ההסתברות שמתוך k שנבחר יהיה לפחות התנגשות אחת?

$$\epsilon = 1 - e^{-\frac{k(k-1)}{2N}} \rightarrow k \cong \sqrt{2N \cdot \ln\left(\frac{1}{1-\epsilon}\right)}$$

ההסתברות שפריטים לא מתנגשים: $P(x) = 1 - e^{-\frac{k^2}{2N}}$



Iterated Hash Function: הקלט מחולק ל- n בלוקים קבועים: $X = \{x_1, x_2, \dots, x_n\}$

(הבלוק האחרון מקבל padding במידת הצורך)

הבלוק הראשון מקבל Initialize vector, כל בלוק עובר דרך פונקציית hash ועם התוצאה של הבלוק הקודם

חתימות דיגיטליות : Digital Signatures

סוג של הצפנה אסימטרית המדמה חתימה על מסמך

דרישות טכניות:

-חתימה תלויה בהודעה עליה חותמים (לכל הודעה חתימה שונה)

-ייחודית לשולח למניעת הונאה

-קל לייצור

-קל לבדיקה ולאישור

-קשה מאד לזיוף

-קל לשמירה

דרישות מערכתיות:

Authentication : דרישה שמי שמקבל את ההודעה רוצה להיות בטוח במי ששלח

Data Integrity :שלמות- דרישה שההודעה תישאר שלמה

Non Repudiation : אי הכחשה – ברגע שבוצעה החתימה אי אפשר להתחרט וידוע שאלים זאת שחתמה

איך חתימות דיגיטליות עובדות:

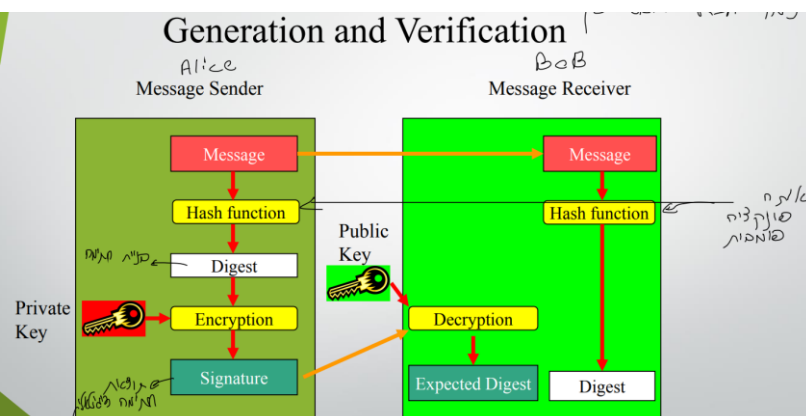
-משתמש A יוצר מפתח פרטי ומפתח פומבי, באמצעות המפתח הפרטי הוא חותם על המסמך

- שליחה של המסמך החתום עם מפתח פומבי לפענוח

-משתמש B שמקבל את ההודעה החתומה יכול לפענח אותה באמצעות המפתח הפומבי

* כמו RSA אבל הפוך

ז"א רק השולח יכול לייצר את החתימה כל השאר יכולים להסתכל מה יש בתוכה



שימוש בפונקציית Hash לחתימה : לא נחתום על כל ההודעה כיוון שזה לוקח זמן במקום זה נחתום על ההודעה שעבר פונקציית hash והיא קצרה יחסית, הדבר עוזר במהירות וכן גם בסודיות ההודעה

Digital Certificates: מכילה בתוך מידה על החתימה על ידי גורם מוסמך Certification Authority
El Gamal Digital Signatures: חתימות אל גמל דומות ל Diffie Helman, משתמשות בשדות גבוהים Galois

אבטחה חזקה עקב בעיית ה Discrete Logarithms

- מפתח פרטי משמש לחתימה (לא הצפנה)

- מפתח פומבי משמש לאימות

עקומים אליפטיים Elliptic curves :

משמשים להצפנות אסימטריות (טקסט או מפתח)

לכל משתמש יש מפתח פומבי ומפתח פרטי

- מפתח פומבי משמש להצפנה או אימות (במקרה של חתימה)

- מפתח פרטי משמש לפענוח או לחתימה

עקומים אליפטיים בקריפטולוגיה:

- צריך להגדיר קבוצה אליפטית,

* עבור RSA נשתמש במקום בחזקה במכפלה

הגדרה : עקום אליפטי בקריפטולוגיה מוגדר להיות משוואה מהצורה :

$$y^2 = (x^3 + ax + b) \bmod p$$

כאשר a,b הם פרמטרים שניתן לשנות.

p מספר ראשוני מאד גדול,

ז"א עלינו להגדיר את העקום בו אנו נרצה להשתמש.

דרישות:

a, b על $-4a^3 + 27b^2 \neq 0 \bmod(p)$, $p > 3$ prime

חיבור בעקומים אליפטיים:

כל מה שצריך בעקומים אליפטיים הוא ביצוע פעולות חיבור!

חלוקה למקרים של תוצאת החיבור והגדרתו: צריך להגדיר איך מבצעים

את הפעולה

עבור נקודות : $P, Q \in E$, $P = (x_1, y_1)$, $Q = (x_2, y_2)$ נחלק ל 3 מקרים:

מקרה 1: $x_1 \neq x_2$ (קיים קו ונקודה שלישית) :

$$P + Q = (x_3, y_3)$$

$$x_3 = m^2 - x_1 - x_2$$

$$y_3 = m * (x_1 - x_3) - y_1$$

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

כאשר m זה השיפוע והחלוקה היא הכפלה בהופכי

מקרה 2: $x_1 = x_2$ AND $y_1 = -y_2$

החיבור מוגדר להיות $(x_3, y_3) \in E$

כאשר $(x, y) + (x, -y) = O$

* O זה נקודה באינסוף

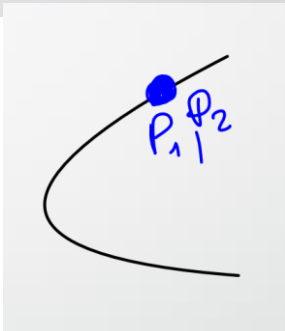
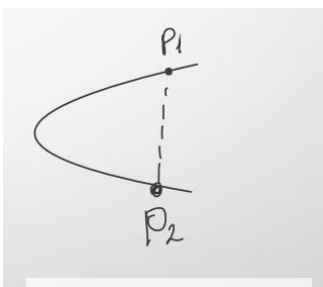
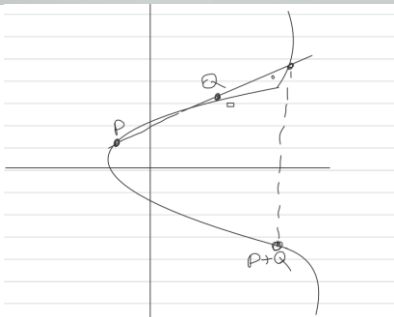
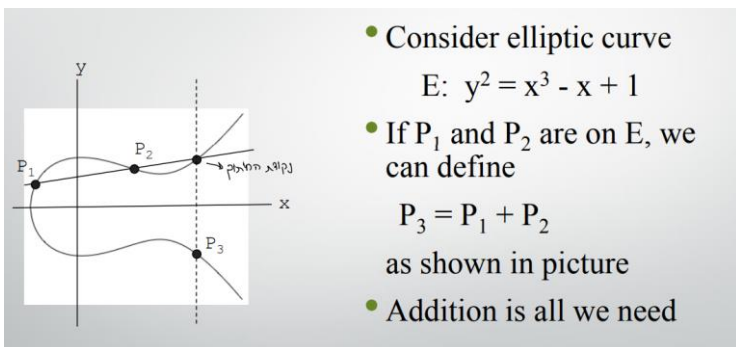
מקרה 3: $x_1 = x_2$ AND $y_1 = y_2$ (אותה הנקודה)

החיבור מוגדר להיות :

$$x_3 = m^2 - x_1 - x_2$$

$$y_3 = m(x_1 - x_3) - y_1$$

$$m = (3x_1^2 + a) * (2y_1)^{-1}$$



הגדרת איבר יחידה בעקומים אליפטיים:

הנקודה O (אינסוף) היא איבר היחידה כך שמתקיים: $P + O = O + P = P$
 ממקרה 2 יש לנו את ההגדרה של ההופכי לנקודה (y -ים הפוכים בסימן ו- x -ים שווים)
 עקומים אליפטיים הם עבור p ראשוני ולכן הם יוצרים חבורה ציקלית. משמע קיים להם יוצר כלשהו (כל נקודה פרט לאינסוף)

העברת מפתח בעזרת עקומים אליפטיים בצורת D-H :

-אליס ובוב רוצים לשתף מפתח משותף
 -אליס מחשבת מפתח פרטי: $Private\ key = a$ ($a \in [1, p-1]$ integer)
 והיא מחשבת מפתח פומבי: $Public\ key = P_A(point) = a * B$, B is base point (creator)
 -בוב מבצע אותו דבר כמו אליס רק $private\ key = b$, $public\ key = P_B = b * B$
 -אליס ובוב מעבירים ביניהם את המפתחות הפומביים
 -אליס מחשבת: $K_{AB} = a * P_B = a(bB) = (a + a + a + \dots a)_b\ times B$
 -כנ"ל בוב (בגלל אסוציאטיביות נקבל את אותו חישוב)
 -קיבלנו מפתח משותף סודי: $K_{AB} = abB$

למה להשתמש בעקומים אליפטיים בקריפטולוגיה:

-עקומים אליפטיים מסתמכים על הקושי בפתירת הבעיה של הלוגריתם הדיסקרטי

- To **protect** a 128 bit AES key it would take a:
 - RSA Key Size: 3072 bits → 512
 - ECC Key Size: 256 bits
- How do we strengthen RSA?
 - Increase the key length

ECC KEY SIZE (Bits)	RSA KEY SIZE (Bits)	KEY SIZE RATIO	AES KEY SIZE (Bits)
163	1024	1 : 6	
256	3072	1 : 12	128
384	7680	1 : 20	192
512	15 360	1 : 30	256

Supplied by NIST to ANSI X9F1

יתרונות עקומים אליפטיים :

- מפתחות קצרים
- חישובים קצרים
- מהיר יותר מ-RSA

Summary of ECC

- “**Hard problem**” analogous to discrete log
 - $Q=kP$, where Q, P belong to a prime curve
 - given $k, P \rightarrow$ “easy” to compute Q
 - given $Q, P \rightarrow$ “hard” to find k
 - known as the **elliptic curve logarithm problem**
 - k must be large enough
- ECC security relies on elliptic curve logarithm problem
 - compared to factoring, can use much smaller key sizes than with RSA etc
 - **for similar security ECC offers significant computational advantages**